

## Esercizio A1

**Limite superiore** Andando per tentativi, proviamo con  $O(n)$ . Proviamo quindi a dimostrare che  $T(n) = O(n)$ .

- Ipotesi induttiva:  $T(k) \leq ck$ , per  $k < n$
- Passo induttivo:

$$\begin{aligned} T(n) &= T(\lfloor n/2 \rfloor) + T(\lfloor n/4 \rfloor) + T(\lfloor n/8 \rfloor) + 2T(\lfloor n/16 \rfloor) + 1 \\ &\leq c\lfloor n/2 \rfloor + c\lfloor n/4 \rfloor + c\lfloor n/8 \rfloor + 2c\lfloor n/16 \rfloor + 1 \\ &\leq \frac{1}{2}cn + \frac{1}{4}cn + \frac{1}{8}cn + \frac{1}{8}cn + 1 \\ &= cn + 1 \leq cn \end{aligned}$$

L'ultima disequazione è falsa, ma per un termine di ordine inferiore.

Proviamo quindi a dimostrare che

$$\exists b > 0, \exists c > 0, \exists m \geq 0 : T(n) \leq cn - b, \forall n \geq m$$

- Ipotesi induttiva:  $T(k) \leq ck - b$ , per  $k < n$
- Passo induttivo:

$$\begin{aligned} T(n) &= T(\lfloor n/2 \rfloor) + T(\lfloor n/4 \rfloor) + T(\lfloor n/8 \rfloor) + 2T(\lfloor n/16 \rfloor) + 1 \\ &\leq c\lfloor n/2 \rfloor - b + c\lfloor n/4 \rfloor - b + c\lfloor n/8 \rfloor - b + 2 \cdot (c\lfloor n/16 \rfloor - b) + 1 \\ &\leq \frac{1}{2}cn - b + \frac{1}{4}cn - b + \frac{1}{8}cn - b + \frac{2}{16}cn - 2b + 1 \\ &= cn - 5b + 1 \leq cn - b \end{aligned}$$

L'ultima disequazione è vera per ogni  $c$  e per  $b \geq 1/4$ .

- Caso base:  $T(n) = 1 \leq cn - b$ , per tutti i valori di  $n$  compresi fra 1 e 16, ovvero:

$$c \geq \frac{b+1}{n}, \forall n : 1 \leq n \leq 16$$

I valori  $\frac{b+1}{n}$  sono minori o uguali di  $5/4$ , per  $1 \leq n \leq 16$ ; quindi tutte queste disequazioni sono soddisfatte da  $c \geq 5/4$ .

Abbiamo quindi dimostrato che  $T(n) = O(n)$ , con  $m = 1$  e  $c \geq 5/4$ .

## Esercizio A2

L'esercizio è molto semplice, ma bisogna fare attenzione a gestire bene gli indici.

Utilizzeremo una funzione ricorsiva `firstRec(int[] A, int i, int j, int v)`, richiamata da `first()`, che assume che il valore sia presente nel sottovettore  $A[i \dots j]$  e quindi la dimensione del sottovettore sia almeno 1:

---

```
int first(int [] A, int n, int v)
return firstRec(A, 1, n, v)
```

---

Come caso base, se il sottovettore ha dimensione 1 ( $i == j$ ) e tenuto conto che il valore è sicuramente presente, allora bisogna ritornare  $i$  stesso.

Altrimenti, sono dati due casi:

- Se il valore cercato è più piccolo o uguale ad  $A[m]$ , allora la prima occorrenza del valore cercato si troverà sicuramente nel sottovettore di sinistra  $A[i \dots m]$  ( $m$  incluso, in quanto  $A[m]$  può contenere la prima occorrenza di  $v$ ). Per come è calcolato  $m$  (intero inferiore), la dimensione del sottovettore è almeno 1.
- Se il valore cercato è più grande di  $A[m]$ , allora la prima occorrenza si troverà sicuramente nel sottovettore di destra,  $m$  escluso. Per come è calcolato  $m$  (intero inferiore), la dimensione del sottovettore è almeno 1.

---

```
int firstRec(int [] A, int i, int j, int v)
```

---

```

if i == j then
    return i
else
    int m = (i + j) / 2
    if v ≤ A[m] then
        return firstRec(A, i, m, v)
    else
        return firstRec(A, m + 1, j, v)

```

---

Molti studenti hanno proposto questa versione, che fa un controllo in più ma è leggermente più chiara:

---

```
int firstRec(int [] A, int i, int j, int v)
```

---

```

if i == j then
    return i
else
    int m = (i + j) / 2
    if v < A[m] then
        return firstRec(A, i, m - 1, v)
    else if v == A[m] then
        return firstRec(A, i, m, v)
    else
        return firstRec(A, m + 1, j, v)

```

---

Molti studenti hanno proposto una versione (che non riporto qui) in cui quando  $A[m] = v$ , si cerca in  $A[m - 1]$  per vedere se per caso è anche la prima occorrenza. Bisogna però fare attenzione che  $m$  non sia uguale a 1 (primo elemento del vettore). Se scritta bene, ha la stessa complessità delle altre, ma la trovo poco elegante.

Uno studente ha proposto questa versione, che modifica il caso base. L'idea è semplice: se il primo elemento del vettore è pari al valore cercato, ritorniamo quel valore perché è sicuramente la prima occorrenza. Altrimenti, sappiamo che la prima occorrenza si trova nel vettore e lavoriamo come in precedenza. Un po' più difficile da spiegare, ma funziona.

---

```
int firstRec(int [] A, int i, int j, int v)
```

---

```
    if A[i] == v then
        return i
    else
        int m = (i + j) / 2
        if v ≤ A[m] then
            return firstRec(A, i, m, v)
        else
            return firstRec(A, m + 1, j, v)
```

---

La complessità per tutte queste versioni è  $O(\log n)$ , come richiesto.

Si noti che se vi avessi chiesto di realizzare `last()` (la funzione che restituisce l'ultima posizione di un'occorrenza), sarebbe stato facile incorrere in un errore con gli indici. Ve lo lascio come esercizio.

Parecchi studenti hanno utilizzato una ricerca dicotomica normale, trovando un valore  $v$  qualunque, e poi hanno cercato linearmente a sinistra di quel valore - vedi sotto. Se questo algoritmo riceve in input un insieme di valori tutti uguali, analizzerà metà del vettore; quindi la complessità nel caso pessimo sarà  $O(n)$ .

A questi studenti ho assegnato 0% come voto, perché non rispetta le consegne. Altrimenti, scrivere un semplice ciclo **for** sarebbe stato sufficiente (e più rapido).

---

```
int firstRec(int [] A, int i, int j, int v)
```

---

```
    if i == j then
        return i
    else
        int m = (i + j) / 2
        if v < A[m] then
            return firstRec(A, i, m - 1, v)
        else if v == A[m] then
            while m > 0 and A[m] == v do
                m = m - 1
            return m + 1
        else
            return firstRec(A, m + 1, j, v)
```

---

### Esercizio A3

Il testo è volutamente complicato (ma non ambiguo). Alla fine, è facile vedere che il messaggio  $m_e$  ( $m_p$ ) raggiunge un nodo  $u$  al giorno  $t_e = d_e[u]$  ( $t_p = d_p[u]$ ), dove  $d_e[u]$  ( $d_p[u]$ ) è la minima distanza (in numero di archi) fra  $u$  e un nodo inizialmente appartenente a  $PE$  ( $PP$ ).

Per calcolare la distanza di un nodo da un insieme di altri nodi, possiamo prendere ispirazione da un problema simile visto durante le esercitazioni (distanza fra due insiemi di nodi).

Calcolare le distanze è facile, è sufficiente modificare l'algoritmo BFS mettendo inizialmente nella coda tutti i nodi che appartengono ad un partito e settando la loro distanza a zero.

Una volta calcolate le distanze da entrambi i partiti, un nodo resterà indeciso per sempre se e solo se verrà raggiunto dai messaggi nello stesso giorno, ovvero se ha la stessa distanza dai nodi dei due partiti. Il costo dell'algoritmo così proposto è pari a  $O(m + n)$ , corrispondente a due visite in ampiezza e al costo di verificare, per ogni nodo, se la distanza dei partiti è uguale.

---

**int[]** dist(**GRAPH**  $G$ , **int[]**  $status$ , **int**  $party$ )

---

```
QUEUE  $Q$  = Queue()
int[]  $dist$  = new int[1... $G.n$ ]
foreach  $u \in G.V()$  do
    if  $status[u] == party$  then
         $Q.enqueue(u)$ 
         $dist[u] = 0$ 
    else
         $dist[u] = \infty$ 
while not  $Q.isEmpty()$  do
    NODE  $u = Q.dequeue()$ 
    foreach  $v \in G.adj(u)$  do
        if  $dist[v] == \infty$  then
             $dist[v] = dist[u] + 1$ 
             $Q.enqueue(v)$ 
return  $dist$ 
```

---

---

**int** election(**GRAPH**  $G$ , **int[]**  $status$ )

---

```
int[]  $distPE = dist(G, status, PE)$ 
int[]  $distPP = dist(G, status, PP)$ 
int  $tot = 0$ 
for  $u = 1$  to  $G.n$  do
     $tot = tot + \text{iff}(distPE[u] == distPP[u], 1, 0)$ 
return  $tot$ 
```

---

È possibile realizzare una versione in cui viene effettuata una sola visita, invece di due, aggiornando due vettori di distanze contemporaneamente e partendo dall'unione dei nodi  $PE$  e  $PP$ . Ma la complessità resta uguale,  $O(m + n)$ .

Esiste anche la possibilità di "simulare" il processo, con un ciclo che scorre tutti i nodi e tutti gli archi, ripetuto  $n$  volte. La complessità di questo algoritmo è  $O(mn)$ .